CrewAI框架：构建多Agent协作系统

- 来源: https://class.imooc.com/lesson/5293#mid=105157
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: CrewAI 实战
- 课节: 图文 1-1 CrewAI框架：构建多Agent协作系统

1-1 CrewAI框架：构建多Agent协作系统

CrewAI框架：构建多Agent协作系统

> [!TIP] 阅读指南 在上一节中，我们深入了解了Google A2A协议——一个为Agent间通信建立标准的协议。但仅有通信标准还不够，我们还需要一个高效的框架来编排和管理多个Agent的协作。 举个场景：你要组织一场大型活动，需要策划、执行、宣传等多个团队协同工作。如果只靠口头协调，效率会很低。你需要项目管理工具来分配任务、跟踪进度、协调资源。 CrewAI就是这样一个"项目管理工具"，专门用于管理多Agent系统的协作。它让Agent团队能够： 明确各自的角色和职责 按照预定义的流程执行任务 自动处理任务依赖和数据传递 监控执行状态并处理异常 本节将从理论出发，系统性地介绍CrewAI框架的核心概念和工作机制，并通过教育场景的实例来加深理解。当然，只看本小节肯定不能很好的理解。建议结合后续的实战代码一起理解。

为什么需要CrewAI框架

多Agent协作的根本挑战

在构建多Agent系统时，会面临几个根本性的挑战：

依赖管理复杂性：多个Agent之间往往存在复杂的依赖关系。一个Agent的输出可能是另一个Agent的输入，手动管理这些依赖既繁琐又容易出错。

执行顺序控制：不同任务需要按照特定的顺序执行，某些任务必须等待前置任务完成后才能开始，这种时序控制需要精确的协调机制。

状态跟踪困难：在复杂的协作流程中，很难实时跟踪每个Agent的执行状态、进度和中间结果。

错误传播处理：当某个Agent执行失败时，如何优雅地处理错误并向其他Agent传达，避免整个系统崩溃。

传统解决方案的局限

```
# 传统手动编排的局限性
def manual_orchestration():
    # 步骤1：执行Agent A
    result_a = agent_a.run(input_data)

    # 步骤2：基于A的结果执行Agent B
    result_b = agent_b.run(result_a)

    # 步骤3：基于B的结果执行Agent C
    result_c = agent_c.run(result_b)

    # 错误处理需要在每个步骤中重复
    # 依赖关系需要手动维护
    # 执行状态难以统一监控
```

这种方式的主要问题：

- 代码冗余：每个项目都需要重复编写类似的编排逻辑

代码冗余

- 维护困难：流程变更需要修改大量代码

维护困难

- 扩展性差：增加新的Agent或调整流程非常困难

扩展性差

- 缺乏标准化：不同项目采用不同的编排方式

缺乏标准化

CrewAI的核心价值

CrewAI框架通过声明式的方式来解决上述根本性问题，让多Agent协作变得简单而优雅：

```
# CrewAI的简洁之美 - 三行代码搞定复杂协作
from crewai import Crew, Agent, Task

# 只需定义角色、任务和依赖关系
crew = Crew(
    # 定义2个Agent
    agents=[Agent(role="分析师"), Agent(role="写手")],
    tasks=[
        # 任务1由Agent0来执行
        Task(description="分析数据", agent=agents[0]),
        # 任务2由Agent1来执行
        Task(description="撰写报告", agent=agents[1], context=[tasks[0]])
    ]
)

# 开始执行，自动处理所有复杂性
result = crew.kickoff()
```

这就是CrewAI的魅力：声明即执行。开发者只需描述"要做什么"和"谁来做"，框架自动处理：

- 任务调度和执行顺序
- 数据在Agent间的传递
- 错误处理和状态监控
- 执行日志和进度跟踪

相比传统手动编排需要数十行代码处理各种细节，CrewAI让复杂协作变得如此简单。

CrewAI核心概念

理解CrewAI就像理解一个现代化的企业组织架构：

```
公司(Crew)
  ↳ 多个部门团队协同工作
——————————————————————————
  部门员工(Agent)  ↔  专业角色
    ↳ 各司其职，有明确的职责和能力
——————————————————————————
  工作任务(Task)   ↔  具体工作
    ↳ 定义做什么、谁来做、怎么做
——————————————————————————
  运行模式(Process) ↔ 管理机制
    ↳ 决策流程和协作方式
```

CrewAI框架正是基于这样的组织思维设计的：

- Crew（团队）：多Agent协作的基本组织单位

Crew（团队）

- Agent（员工）：具有特定角色和能力的执行单元

Agent（员工）

- Task（任务）：定义具体工作内容和执行标准

Task（任务）

- Process（流程）：控制任务执行的管理模式

Process（流程）

让我们逐一深入理解这四个核心概念：

Crew（团队）- 协作的组织单位

理论定义：Crew是CrewAI中多Agent协作的基本组织单位，类似于现实世界中的项目团队。它将多个具有不同专业能力的Agent组织在一起，按照预定义的流程协同完成复杂任务。

核心组成要素：

- Agents集合：多个具有不同角色和专长的Agent

Agents集合

- Tasks序列：预定义的工作任务流程

Tasks序列

- Process模式：任务执行的控制模式（顺序或层级）

Process模式

- 配置参数：执行环境和行为控制选项

配置参数

```
# 创建一个Crew
basic_crew = Crew(
    agents=[agent1, agent2, agent3],  # Agent团队
    tasks=[task1, task2, task3],      # 任务序列
    process=Process.sequential,       # 执行模式
    verbose=True                      # 日志级别
)
```

设计原则：

设计原则

- 每个Crew应该有明确的业务目标
- Agent的技能组合应该覆盖任务所需的所有专业领域
- 任务流程应该反映实际的业务逻辑依赖关系

Agent（智能体）- 执行单元

理论定义：Agent是Crew中的基本执行单元，代表具有特定角色、目标和能力的智能实体。每个Agent都是一个独立的决策主体，能够在给定的约束条件下自主完成分配的任务。

核心属性：

- Role（角色）：Agent的身份标识和专业定位

Role（角色）

- Goal（目标）：Agent需要达成的具体目标

Goal（目标）

- Backstory（背景）：影响Agent行为模式的专业背景

Backstory（背景）

- Tools（工具）：Agent可用的外部能力集合（非常重要）

Tools（工具）

- LLM配置：底层语言模型的配置参数

LLM配置

```
# Agent的完整定义
professional_agent = Agent(
    role="数据分析师",                    # 专业角色
    goal="从数据中提取有价值的洞察",       # 具体目标
    backstory="拥有10年数据分析经验的专业人士", # 专业背景
    tools=[data_processor, visualization], # 专业工具
    llm=ChatOpenAI(model="gpt-5")        # AI能力配置
)
```

行为特征：

行为特征

- 基于角色和目标做出决策
- 能够使用工具扩展自身能力
- 在任务执行中保持一致性

Task（任务）- 工作单元

理论定义：Task是Crew中最小的工作单元，定义了具体需要完成的工作内容、负责的Agent以及期望的输出结果。Task之间可以通过依赖关系形成复杂的工作流。

核心要素：

- Description（描述）：详细的工作内容说明

Description（描述）

- Agent（负责人）：负责执行此任务的Agent

Agent（负责人）

- Expected Output（期望输出）：任务完成标准和输出格式

Expected Output（期望输出）

- Context（上下文）：依赖的前置任务结果

Context（上下文）

- Tools（专用工具）：此任务可使用的特定工具

Tools（专用工具）

```
# Task的完整定义
analysis_task = Task(
    description="分析用户行为数据，识别关键趋势和异常模式",
    agent=data_analyst,                    # 指定执行Agent
    expected_output="包含趋势图表和洞察结论的分析报告",
    context=[data_collection_task],        # 依赖前置任务
    tools=[statistical_analyzer]           # 专用分析工具
)
```

依赖机制：

依赖机制

- Context属性建立任务间的依赖关系
- 前置任务的输出自动成为后续任务的输入
- 支持多对多的复杂依赖拓扑

> [!TIP] 大多数时候，一个复杂的任务会由多个Task构成。这些Task之间会形成工作流，具体的组织结构由后续的Process来决定。

Task与Agent的区别 - 常见混淆点

很多初学者容易混淆Task和Agent，这里我们明确两者的本质区别：

Agent是"谁"，Task是"做什么"：

Agent是"谁"，Task是"做什么"

```
# Agent：定义能力和角色
analyst = Agent(
    role="数据分析师",           # 我是谁
    goal="提供数据洞察",         # 我的职责
    tools=[data_tool, viz_tool]  # 我能用什么工具
)

# Task：定义具体工作
task = Task(
    description="分析本月销售数据",  # 做什么
    agent=analyst,                  # 谁来做
    expected_output="分析报告"      # 要什么结果
)
```

简单来说，Agent可以理解成员工，Agent通常会定义员工的能力，但并不涉及他具体做什么；但Task不同，Task（任务）就是明确要做的事情，是业务。 比如有A员工擅长Java Web开发；B员工擅长Python数据分析，这就是Agent，他们并不涉及具体的任务。 而Task类似我要用Java语言开发一个Web网站，那么很显然A员工就是合适的完成此任务的Agent。

核心差异对比：

核心差异对比

维度	Agent（能力主体）	Task（工作指令）
定位	专业人员	具体任务
定义内容	角色、目标、工具	工作内容、标准
可复用性	可执行多个Task	一次性工作
生命周期	Crew存续期间持续存在	执行完成后结束

容易混淆的场景：

容易混淆的场景

场景1：Agent有tools，Task也有tools?

- Agent的tools：这个Agent通常可以使用的工具集合
- Task的tools：限制此任务只能使用这些特定工具

```
# Agent级别的tools - 通用能力
analyst = Agent(role="分析师", tools=[general_tool, data_tool, viz_tool])

# Task级别的tools - 限制工具范围
task = Task(
    description="分析销售数据",
    agent=analyst,
    tools=[data_tool]  # 执行这个任务时，只能用data_tool
)
```

工具使用规则：

工具使用规则

- 如果Task指定了tools：Agent只能使用Task指定的这些工具（即使Agent有其他工具也不能用）
- 如果Task没指定tools：Agent可以使用自己的所有工具

这种机制的好处是可以针对特定任务限制工具使用范围，提高安全性和执行效率。

场景2：一个Agent可以执行多个Task吗？ 可以！一个Agent就像一个员工，可以承担多项工作：

```
analyst = Agent(role="数据分析师")

task1 = Task(description="分析用户数据", agent=analyst)
task2 = Task(description="分析销售数据", agent=analyst)
task3 = Task(description="生成月度报告", agent=analyst)
# 同一个Agent执行三个不同的任务
```

场景3：一个Task可以分配给多个Agent吗？ 不可以！CrewAI中Task和Agent是1:1绑定关系，一个Task只能指定一个Agent负责执行。

```
# ✗ 错误：Task不能分配给多个Agent
task = Task(
    description="复杂分析",
    agent=[analyst1, analyst2]  # 这样是不行的
)

# ✓ 正确：需要拆分成多个Task
task1 = Task(description="数据收集", agent=analyst1)
task2 = Task(description="数据分析", agent=analyst2, context=[task1])
```

Agent是人，Task是任务或者工单；人可以干多个活，但每个工单只能一个人负责。

Process（执行模式）

之前讨论过，完成一项复杂的工作可能需要多个任务（Task）。那么多个任务之间是什么关系？Process就定义了Crew中多个Task的执行模式，决定了任务如何被调度和执行。

CrewAI提供两种主要的执行模式来适应不同的协作需求。理解这两种模式的差异，对于设计高效的多Agent系统至关重要。

Sequential（顺序模式）

顺序模式是最简单直观的执行方式，任务严格按照定义的顺序依次执行。

核心特点：

- 任务严格按照定义顺序执行
- 每个任务必须等待前置任务完成后才能开始
- 前一个任务的输出自动成为下一个任务的上下文
- 适用于线性依赖的工作流程

代码示例：

代码示例

```
from crewai import Crew, Agent, Task, Process

# 定义三个Agent
analyzer = Agent(role="数据分析师", goal="分析数据")
designer = Agent(role="方案设计师", goal="设计方案")
generator = Agent(role="内容生成器", goal="生成内容")

# 定义三个任务，形成流水线
analysis = Task(description="分析用户需求", agent=analyzer)
design = Task(description="基于分析结果设计方案", agent=designer)
generation = Task(description="基于设计方案生成内容", agent=generator)

# 顺序执行模式
sequential_crew = Crew(
    agents=[analyzer, designer, generator],
    tasks=[analysis, design, generation],
    process=Process.sequential  # 严格的顺序执行
)

# 执行：analysis → design → generation
result = sequential_crew.kickoff()
```

执行流程：

执行流程

```
Task 1 (分析)  →  完成  →  输出传递
                           ↓
Task 2 (设计)  →  完成  →  输出传递
                           ↓
Task 3 (生成)  →  完成  →  最终结果
```

> [!NOTE] 为什么Crew需要同时指定agents和tasks？ 你可能注意到，虽然每个Task已经指定了agent（如task = Task(agent=analyst)），但创建Crew时仍需要传入agents列表。这是因为：资源池管理：Crew需要知道有哪些Agent可用，统一管理它们的生命周期和资源 全局配置：Crew可以为所有Agent设置统一的配置（如LLM、memory、max_rpm等） Hierarchical模式：在层级模式下，Manager需要知道整个团队有哪些成员可以调配 简单理解：Task.agent指定"谁来做这个任务"，Crew.agents定义"团队有哪些成员"。

Hierarchical（层级模式）

层级模式引入了一个Manager Agent来协调整个团队，这是一种更灵活、更智能的执行方式。

> [!IMPORTANT] Manager的本质：Manager本质上是一个特殊的Agent，负责协调和管理其他Agent。它可以通过两种方式创建： 自动创建：通过manager_llm参数，CrewAI会自动创建一个Manager Agent 自定义：通过manager_agent参数，使用你自己定义的Manager Agent Manager Agent和普通Agent的区别在于职责：普通Agent负责执行具体任务，Manager Agent负责任务的规划、分配和监控。

核心特点

- 引入Manager Agent作为协调者
- Manager负责任务分解、分配和监控
- 支持动态任务分配和并行处理
- 其他Agent作为执行者，专注于各自的专业领域

工作机制：

在层级模式下，执行流程发生了本质变化：

- Manager接管控制权：不再是简单的顺序执行，而是由Manager统一调度

Manager接管控制权

- 动态任务分解：Manager可以将复杂任务分解为多个子任务

动态任务分解

- 智能分配：Manager根据Agent的角色和能力，动态分配任务

智能分配

- 结果整合：Manager收集各个Agent的执行结果，进行整合和协调

结果整合

```
from crewai import Crew, Agent, Task, Process
from langchain_openai import ChatOpenAI

# 定义专业Agent（不需要定义Manager）
researcher = Agent(
    role="研究员",
    goal="深入研究特定主题",
    backstory="经验丰富的研究专家"
)

analyst = Agent(
    role="分析师",
    goal="分析研究数据",
    backstory="数据分析专家"
)

writer = Agent(
    role="写手",
    goal="撰写专业报告",
    backstory="专业技术写作者"
)

# 定义任务（不需要指定具体的agent）
research_task = Task(
    description="研究人工智能最新发展",
    expected_output="详细的研究报告"
)

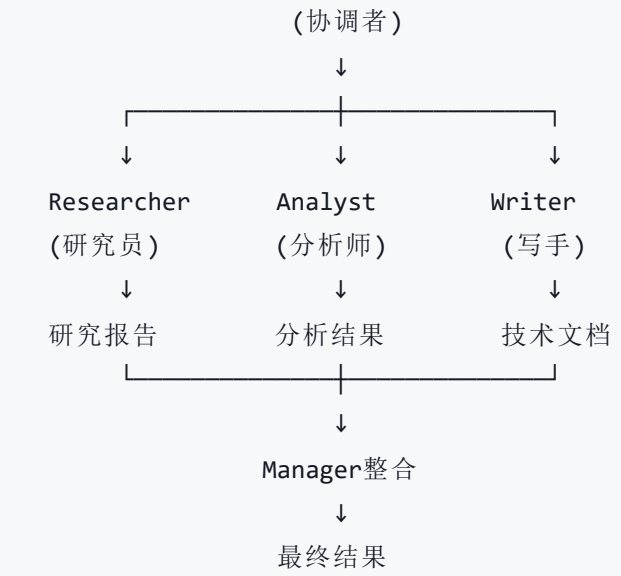
analysis_task = Task(
    description="分析研究数据并提取关键洞察",
    expected_output="数据分析结果"
)

writing_task = Task(
    description="撰写综合技术报告",
    expected_output="完整的技术报告"
)

# 层级执行模式
hierarchical_crew = Crew(
    agents=[researcher, analyst, writer],
    tasks=[research_task, analysis_task, writing_task],
    process=Process.hierarchical,
```

```
    manager_llm=ChatOpenAI(model="gpt-4")  # Manager使用的LLM
)

# 执行：由Manager协调整个流程
result = hierarchical_crew.kickoff()
```



关键差异：

关键差异

维度	Sequential模式	Hierarchical模式
控制方式	预定义顺序	Manager动态调度
Task分配	提前绑定Agent	Manager动态分配
执行方式	严格串行	可能并行
灵活性	固定流程	动态调整
适用场景	简单线性流程	复杂协调任务

选择建议：

选择建议

- 优先选择Sequential：如果任务流程清晰、依赖关系明确

优先选择Sequential

- 考虑Hierarchical：如果需要动态协调、并行处理或复杂决策

考虑Hierarchical

- 渐进升级：可以从Sequential开始，遇到瓶颈时再升级到Hierarchical

CrewAI执行机制详解

数据传递机制

CrewAI提供多种数据传递方式：

直接传递：

直接传递

```
# 前置任务的输出直接作为后续任务的输入
task_b = Task(
    description="处理分析结果",
    agent=processor,
    context=[task_a]  # 自动获取task_a的输出
)
```

结构化传递：

结构化传递

```
# 通过特定字段传递数据
task_c = Task(
    description="生成报告",
    agent=reporter,
    context={
        "analysis_data": task_a.output,
        "processing_results": task_b.output
    }
)
```

错误处理机制

CrewAI内置完善的错误处理体系，可以通过配置参数来控制异常情况下的行为：

```
# 错误处理配置
robust_crew = Crew(
    agents=[agent1, agent2],
    tasks=[task1, task2],
    process=Process.sequential,
    max_rpm=10,        # 速率限制：每分钟最多10次请求
    max_retries=3,     # 最大重试次数：失败后最多重试3次
    retry_delay=60     # 重试间隔：每次重试间隔60秒
)
```

参数详解：

参数详解

max_rpm（Rate Per Minute，每分钟请求数）：

max_rpm（Rate Per Minute，每分钟请求数）

- 作用：限制对LLM API的调用频率，防止超出API提供商的速率限制

作用

- 场景：当使用有速率限制的API时（如OpenAI的免费层级有RPM限制）
- 示例：max_rpm=10表示每分钟最多调用10次LLM API

示例

max_retries（最大重试次数）：

max_retries（最大重试次数）

- 作用：当任务执行失败时，自动重试的最大次数
- 示例：max_retries=3表示失败后会自动重试最多3次，加上首次执行共4次尝试
- 建议：一般设置2-3次，避免无限重试浪费资源

retry_delay（重试延迟，单位：秒）：

retry_delay（重试延迟，单位：秒）

- 作用：两次重试之间的等待时间
- 场景：给API服务恢复时间，避免立即重试导致连续失败
- 示例：retry_delay=60表示失败后等待60秒再重试
- 建议：根据错误类型调整，API限流建议60秒以上，网络问题可以短一些（10-30秒）

日志与调试配置

在开发和调试CrewAI应用时，详细的日志输出非常重要。CrewAI提供了verbose参数来控制日志的详细程度。

verbose参数详解

基本配置：

基本配置

```
from crewai import Crew, Agent, Task, Process

# 不同的verbose级别
crew = Crew(
    agents=[agent1, agent2],
    tasks=[task1, task2],
    process=Process.sequential,
    verbose=True  # 或者 False, 0, 1, 2
)
```

verbose参数值说明：

verbose参数值说明

参数值	含义	输出内容
False 或 0	静默模式	只输出最终结果，不显示执行过程
True 或 1	标准模式	显示任务执行的关键步骤和结果
2	详细模式	显示完整的执行过程，包括工具调用、中间结果等

不同模式的输出对比：

不同模式的输出对比

verbose=False（静默模式）：

verbose=False（静默模式）

```
crew = Crew(
    agents=[analyst],
    tasks=[analysis_task],
    verbose=False
)
result = crew.kickoff()
# 输出：只有最终结果
# "数据分析完成：销售额增长15%"
```

verbose=True（标准模式）：

verbose=True（标准模式）

```
crew = Crew(
    agents=[analyst],
    tasks=[analysis_task],
    verbose=True
)
result = crew.kickoff()
# 输出：
# > Entering new CrewAgentExecutor chain...
# > Task: 分析销售数据
# > Agent: 数据分析师 正在工作...
# > Task完成：数据分析完成：销售额增长15%
```

verbose=2（详细模式）：

verbose=2（详细模式）

```
crew = Crew(
    agents=[analyst],
    tasks=[analysis_task],
    verbose=2
)
result = crew.kickoff()
# 输出：
# > Entering new CrewAgentExecutor chain...
# > Task: 分析销售数据
# > Agent: 数据分析师 正在工作...
# > 工具调用：data_analysis_tool
#   输入：{"data": "sales_data.csv", "metric": "revenue"}
# > 工具输出：{"result": "revenue_increased", "percentage": 15}
# > Agent思考：根据分析结果，销售额确实增长了...
# > 最终答案：数据分析完成：销售额增长15%
# > Task完成
```

使用建议：

使用建议

- 开发调试阶段：使用verbose=2 可以看到完整的执行流程 帮助定位问题和优化Prompt 理解Agent的决策过程
- 测试验证阶段：使用verbose=True 平衡信息量和可读性 关注关键步骤的执行情况 验证任务流程是否正确
- 生产环境：使用verbose=False 减少不必要的日志输出 提高执行效率 配合专业的日志系统使用

注意事项：

注意事项

- verbose参数不影响程序的执行结果，只影响日志输出
- 详细模式会输出大量信息，可能影响控制台可读性
- 在生产环境建议关闭verbose，使用专门的日志系统记录关键信息
- 如果需要保存执行日志，可以使用output_log_file参数将日志输出到文件

下一节预告

掌握了CrewAI框架的理论基础和核心概念后，下一节我们将进入CrewAI实战进阶。 这将带你从理论学习走向真正的项目实战，构建能够解决实际业务问题的智能系统。